

Uganda Martyrs University

Web based programming project

Group 5

Names	Reg number
Kiiza wilbrod	2024-B071-22937
Bakulumpagi Ronald	2024-B071-21433
Lunkuuse Priscilla Angel	2024-B071-22934
Mwagaza Latif	2024-B071-21434

ONLINE VOTING SYSTEM

Project Report for CSC 2202: Web Based Systems Programming
Uganda Martyrs University – Semester 2, 2025/2026

1. SYSTEM OVERVIEW

The Online Voting System is a web-based application developed to facilitate secure and efficient student guild elections within the university. The system enables registered users to vote electronically for candidates while allowing administrators to manage candidates, monitor voting activities, and control election status.

The system improves transparency, minimizes manual voting errors, and simplifies election management through automation and centralized data storage.

PROJECT OBJECTIVES

- Enable users to register and log in securely
- Allow students to vote for candidates electronically
- Prevent duplicate voting using database constraints
- Allow administrators to add, edit, and delete candidates
- Enable image upload for candidates
- Provide vote tally and voting statistics
- Implement role-based access control
- Allow administrators to open and close voting sessions
- Maintain persistent and secure database storage

TECHNOLOGIES USED

- Frontend: HTML5, CSS3, Bootstrap 5
- Backend: PHP 7.4+
- Database: MySQL 5.7+
- Server: Apache (XAMPP)

2. SYSTEM ARCHITECTURE & DATABASE STRUCTURE

DATABASE SCHEMA

TABLE: users

- id (INT, PK, AI)
- name (VARCHAR 100)
- email (VARCHAR 100, UNIQUE)
- password (VARCHAR 255, BCRYPT hashed)
- role (VARCHAR 50)
- has_voted (BOOLEAN)
- created_at (TIMESTAMP)
- updated_at (TIMESTAMP)

TABLE: candidates

- id (INT, PK, AI)
- name (VARCHAR 100)
- description (TEXT)
- image (VARCHAR 255)
- created_by (INT, FK → users.id)
- created_at (TIMESTAMP)

TABLE: votes

- id (INT, PK, AI)
- user_id (INT, FK → users.id)
- candidate_id (INT, FK → candidates.id)
- voted_at (TIMESTAMP)
- UNIQUE KEY (user_id)

TABLE: voting_config

- id (INT, PK, AI)
- voting_open (BOOLEAN)

- updated_by (INT, FK → users.id)
- updated_at (TIMESTAMP)

RELATIONSHIPS

- One administrator can create many candidates (1:N)
- One user can cast only one vote (1:1)
- One candidate can receive many votes (1:N)
- Voting configuration is controlled by administrators
- Unique constraint prevents duplicate voting

3. FUNCTIONAL REQUIREMENTS IMPLEMENTATION

REQUIREMENT 1: User Registration and Login

Implementation:

register.php allows users to create accounts using name, email, and password. Passwords are encrypted using bcrypt hashing. login.php authenticates users and creates session variables after successful login.

REQUIREMENT 2: User Logout and Session Control

Implementation:

logout.php destroys active sessions and redirects users to the login page. Protected pages verify \$_SESSION['id'] before granting access.

REQUIREMENT 3: Data Entry Through Forms

Implementation:

Administrators can add candidates using forms that include candidate name, description, and image upload fields. Data is inserted into the database using POST requests and prepared statements.

REQUIREMENT 4: Input Validation with Feedback

Implementation:

The system validates:

- Empty fields
- Email format
- Password confirmation

- Password length
- Image file types

Error and success messages are displayed using Bootstrap alerts.

REQUIREMENT 5: Dynamic Data Display

Implementation:

Candidate information, vote counts, and user details are dynamically retrieved from the MySQL database and displayed using PHP loops and SQL queries.

REQUIREMENT 6: Search or Filter

Implementation:

The admin dashboard and voter dashboard include candidate search functionality using SQL LIKE queries.

REQUIREMENT 7: Editing Existing Records

Implementation:

Administrators can edit candidate details and user accounts through pre-filled forms. Changes are updated using SQL UPDATE queries.

REQUIREMENT 8: Deleting Records

Implementation:

Administrators can delete candidates and users. Related votes are also removed when necessary to maintain database consistency.

REQUIREMENT 9: Role-Based Access Control

Implementation:

Two user roles exist:

- admin
- user

Only administrators can:

- Manage candidates
- Edit/delete users
- Open/close voting
- View vote tally

Regular users can only vote and view candidates.

REQUIREMENT 10: Persistent Data Storage

Implementation:

All information is stored in a MySQL database with foreign key constraints and timestamps. Data remains available after server restarts and page refreshes.

4. KEY FEATURES & SCREEN DESCRIPTIONS

SCREEN 1: Homepage (index.php)

Display:

- Welcome page
- Navigation links
- Login/Register buttons
- Voting system introduction

Accessibility:

- Public

SCREEN 2: Registration Page (register.php)

Display:

- Full Name field
- Email field
- Password field
- Confirm Password field
- Register button

Validation:

- Email uniqueness
- Password confirmation
- Minimum password length

Accessibility:

- Public

SCREEN 3: Login Page (login.php)

Display:

- Email field
- Password field
- Login button

Accessibility:

- Public

SCREEN 4: User Dashboard (dashboard.php)

Display:

- Candidate cards
- Candidate images
- Candidate descriptions
- Vote button
- Search functionality
- Current vote counts

Features:

- One vote per user
- Voting confirmation popup
- Voting closed/open notifications

Accessibility:

- Registered users only

SCREEN 5: Admin Dashboard (Admin/index.php)

Display:

- Vote tally table
- Candidate management table
- User management table

- Voting control section
- Candidate search bar

Features:

- Add candidate
- Edit candidate
- Delete candidate
- Upload candidate images
- Edit users
- Delete users
- Open/close voting

Accessibility:

- Admin only

SCREEN 6: Add/Edit Candidate Forms

Display:

- Candidate name field
- Description textarea
- Image upload input
- Submit buttons

Validation:

- Required name field
- Image type restrictions

Accessibility:

- Admin only

5. SYSTEM WORKFLOW

USER WORKFLOW

1. User visits homepage

2. User registers an account
3. User logs into the system
4. User accesses dashboard
5. User views candidate information
6. User selects one candidate
7. User submits vote
8. Vote is stored in database
9. System prevents duplicate voting
10. User receives confirmation message

ADMIN WORKFLOW

1. Admin logs into system
2. Admin accesses admin dashboard
3. Admin adds candidates with images
4. Admin edits or deletes candidates
5. Admin manages user accounts
6. Admin opens or closes voting session
7. Admin monitors vote tally in real time
8. Admin logs out

6. CHALLENGES ENCOUNTERED & LESSONS LEARNED

CHALLENGES ENCOUNTERED

Challenge 1: Preventing Duplicate Votes

Issue:

Users could potentially vote multiple times.

Solution:

Added UNIQUE constraint on user_id in the votes table and checked voting status before vote insertion.

Lesson:

Database constraints are important for enforcing system integrity.

Challenge 2: Session Management

Issue:

Unauthorized users accessed protected pages.

Solution:

Implemented session checks on all protected pages.

Lesson:

Always validate sessions before granting access.

Challenge 3: File Upload Handling

Issue:

Image uploads caused invalid file errors.

Solution:

Restricted uploads to JPG, PNG, JPEG, and GIF formats.

Lesson:

Always validate uploaded files for security and compatibility.

Challenge 4: Role-Based Access Control

Issue:

Normal users could access admin pages directly through URLs.

Solution:

Added role verification using `$_SESSION['role']`.

Lesson:

Security checks must exist both in the interface and backend.

Challenge 5: Database Relationships

Issue:

Deleting users caused orphaned vote records.

Solution:

Votes linked to deleted users were removed before deleting the user.

Lesson:

Proper relationship handling prevents database inconsistencies.

7. LESSONS LEARNED

1. Password hashing is essential for user security
2. Prepared statements prevent SQL injection attacks
3. Session management is critical in authentication systems
4. Validation must be performed server-side
5. Database design directly affects system reliability
6. User-friendly interfaces improve usability
7. Security should be implemented from the beginning

8. IMPROVEMENTS FOR FUTURE VERSIONS

- Add election result charts and analytics
- Implement email verification
- Add multiple election categories
- Add voting deadlines and countdown timers
- Implement candidate manifesto uploads
- Add live result visualization
- Export results to PDF/CSV
- Add dark mode
- Implement biometric or student ID verification
- Add audit logs for admin activities

9. CONCLUSION

The Online Voting System successfully implements all required functionalities for a secure and dynamic web application. The system demonstrates:

- ✓ Secure user authentication and session management
- ✓ Complete CRUD operations for candidates and users
- ✓ Dynamic vote tally updates
- ✓ Role-based access control

- ✓ Secure database management using MySQL
- ✓ Input validation and file upload handling
- ✓ Prevention of duplicate voting

The project demonstrates practical application of PHP, MySQL, HTML, CSS, and Bootstrap in developing a real-world web-based election management system for university environments.